

開発環境をCodespacesからローカルへ替える手続き

「GitHub Codespaces」では月ごとに無料で利用できる枠が決まっており、有料プランに加入して従量制の追加利用料を支払わない限り、翌月まで利用が停止されてしまいます。

そのため、無料利用枠を超えて引き続き開発を行うには、予めローカル上のVS CodeでSpringBootを利用できるようにしておく必要があります。

ローカルでのプロジェクト読み込みの準備

プロジェクトをクローンする

Git-Bashなどでローカルの任意スペースをCD指定して、「git clone {このプロジェクトのgit用URL}」を実行することで最新のプロジェクトをローカルにクローンします。

VS CodeでJavaとSpringBootを使えるようにする

VS Codeで拡張機能エリアを表示して、「SpringBoot Extension Pack」をインストールしておきます。

ここからは、既にローカルでJavaの環境設定が施されていることを前提に記述します。

- Javaがインストールされていない、またはシステム環境設定でパス変数が設定されていないようでしたら、事前にご調査の上、設定しておいてください。

ターミナルで `java -version` コマンドを実行することで、Javaのバージョンを確認してください。

- ここから先、ローカルで編集していくこのプロジェクトのJavaバージョンは、ここで出たバージョンに合わせる必要があります。

プロジェクト内のファイルの修正

Codespaces側が2026年5月時点で最新のJava 25で動作していたのに対し、ローカルのJavaは21であると仮定します。

この場合、プロジェクト内の特定ファイルを修正することで、**「Java 25で動かす」という設定を「Java 21で動かす」に書き換える** が必要となります。

1. pom.xml（最重要）

このファイルはMavenプロジェクトの全体設定ファイルです。 backend フォルダの直下にあります。

ファイルを開くと、上部に <properties> というタグでJavaのバージョンが指定されている箇所があります。ここが 25 になっているはずですので、 21 に書き換えます。

```
<properties>  
  <java.version>21</java.version>  
</properties>
```

2. .changeset や .devcontainer（もしあれば）

もし、プロジェクトのルート（または backend フォルダ内）に、VS Code用の設定フォルダである .vscode フォルダがあり、その中に settings.json が存在する場合は、そこも確認してください。

以下のような記述があれば、数値を 21 に変更するか、行ごと削除しても構いません（削除すると、PC全体のデフォルトであるJava 21が自動で使われます）。

```
{
  "java.configuration.runtimes": [
    {
      "name": "JavaSE-21",
      "path": "（お使いのJDK21のパス）",
      "default": true
    }
  ]
}
```

修正した後の「反映手順」

ファイルの記述を書き換えただけでは、VS Codeの拡張機能（Language Support for Java）が古い設定をキャッシュしていて、同じエラーを出し続けることがあります。

記述を直したら、必ず以下の手順で設定をクリーンアップしてください。

- VS CodeのJavaキャッシュをクリアする

- VS Code上で `Ctrl(MacはCmd) + Shift + P` を押し、コマンドパレットを開きます。
- 「Java: Clean Java Language Server Workspace」と入力して選択します。
- 「再起動 (Restart)」を促すポップアップが出るので、指示に従ってVS Codeを再起動します。

- プロジェクトのクリーンと起動

- ターミナルで `backend` フォルダに移動し、念のため過去のビルド残骸を消すために `clean` を挟んで起動コマンドを実行します。

```
cd backend  
mvnw clean spring-boot:run
```

これでSpring Boot側も「Java 21のプロジェクト」として認識し、ローカルのJDK 21を使って正常に起動するようになるはずです。

VS CodeのJava設定（`java.jdt.ls.java.home`）を修正する

念のため、VS Codeの拡張機能に対して使用するJavaの場所を明示的に教えておきましょう。

- VS Codeを開き、設定（ショートカット: `Ctrl + ,`、または `Cmd + ,`）を開きます。
- 検索窓に `java.jdt.ls.java.home` と入力します。
- `Settings.json` で編集 をクリックし、以下のようにインストールしたJDKのパスを指定します。
 - `"java.jdt.ls.java.home": "`（ここに正しいJDKのインストールフォルダへのパスを記述）`"`
 - ※Windowsの場合、パスの区切り文字 `\` は `\\` と二重にする必要があります（例: `C:\\Program Files\\...`）。
- 設定後、VS Codeを一度完全に再起動します。

設定が完了したら、再び `backend` フォルダに移動し、使い慣れたコマンドを実行してみてください。

```
cd backend
mvnw spring-boot:run
```

これでローカルでもCodespaces環境と同じようにSpring Bootが立ち上がるはずです。

フロントエンドのViteに関連するエラーの修正

ターミナルで `cd` を用いて、カレントディレクトリをfrontendに合わせた後、`npm run dev` を実行した際に、`'vite' is not recognized as an internal or external command...` というエラーが出る筈です。

これはWindowsのコマンドプロンプトやターミナルが「Vite（開発サーバーを動かすプログラム）がどこにあるか分からない」と言っている状態です。

根本原因： `node_modules` が存在しない

GitHub Codespaces上では、バックグラウンドや初期設定で自動的にパッケージ（ライブラリ）のインストールが行われていたため、最初から `vite` コマンドが使えていました。

しかし、Git（GitHub）の仕組み上、`node_modules` という数万ものファイルが含まれる依存ライブラリのフォルダは、容量削減や競合防止のためにリモートリポジトリには保存されない（プッシュされない）設定（`.gitignore`）になっているのが一般的です。

そのため、ローカルにクローンしてきた直後のfrontendフォルダ内にはソースコード（`.vue` や `.js` など）しかなく、これらを動かすための実体（Viteなど）がまだダウンロードされていない状態なのです。

解消するための手順

これはローカル環境に必要なパッケージをダウンロードすることで、すぐに解決します。

```
cd frontend  
npm install
```

これにより、`package.json` という設計図を基に、ローカル環境へ `node_modules` フォルダ（Viteなどを含む）が一括ダウンロードされます。

- ※インターネット環境によりますが、数十秒から数分かかる場合があります。
- ※もし「`npm` というコマンド自体が認識されません」と出た場合は、ローカルPCに **Node.js** がインストールされていない可能性があります。その場合は、事前に [Node.js 公式サイト](#) から推奨版（LTS）をインストールしてください。

なぜ個別のインストールではなく `npm install` なのか？

「Viteだけを狙ってダウンロードするコマンド（`npm install vite`）を打てばいいのでは？」と思われるかもしれません。

しかし、それをしない方が良い理由は、プロジェクトの設計図である `package.json` にあります。

frontendフォルダの中にある `package.json` というファイルには、Viteだけでなく、Codespacesで開発していた時に使っていた他の重要なライブラリ（画面をきれいに整えるCSSフレームワークや、通信用のライブラリなど）の「名前」と「正確なバージョン」がすべてリストアップされています。

```
# これだとViteしか入りません（他の必要なライブラリが足りず、別のエラーになります）
```

```
npm install vite
```

```
# これが正解！
```

```
npm install
```

```
# 設計図（package.json）に書かれたViteを含むすべてのライブラリを、  
# Codespacesの時と「寸分狂わぬ同じバージョン」で一括ダウンロードしてくれます。
```

再度起動コマンドを実行する

インストールが完了したら、改めてコマンドを実行してみてください。

```
npm run dev
```

まとめ

- **Backend (Spring Boot)** : `mvnw` (Maven Wrapper) が最初からフォルダに含まれているため、Java のバージョンさえ合えばすぐ動いた。
- **Frontend (Vite / Vue.js)** : 依存ライブラリ (`node_modules`) をローカル側で新しく生成する必要があるため、最初に `npm install` が必須。

これでfrontend側も無事に立ち上がり、ローカル環境だけで完結する開発環境が整うはずです。